



Unidad Didáctica

Instalación y configuración de SQL Server, Unidad I

Contenido

Introducción	2
Objetivos Específicos	2
Orientaciones Específicas	2
Esquema de Trabajo Independiente	2
Actividades de Aprendizaje	2

La administración de bases de datos constituye uno de los pilares fundamentales en la formación de un ingeniero en sistemas de información. SQL Server, como sistema de gestión de bases de datos líder en el mercado empresarial, ofrece un conjunto robusto de herramientas que permiten no solo almacenar información, sino también garantizar su integridad, optimizar su acceso y automatizar procesos mediante programación transaccional.

En esta primera unidad, los estudiantes han adquirido los conocimientos necesarios para instalar y configurar SQL Server, así como para comprender su arquitectura y componentes. La presente evaluación sumativa tiene como propósito llevar estos conocimientos al siguiente nivel: diseñar e implementar una base de datos funcional aplicando criterios profesionales, donde no solo se cumplan los requisitos funcionales, sino que se demuestre capacidad crítica para proponer mejoras, anticiparse a problemas y justificar las decisiones técnicas adoptadas.

A través de un caso práctico contextualizado en una universidad nicaragüense, los estudiantes asumirán el rol de administradores de bases de datos (DBA) y deberán construir una solución que refleje las buenas prácticas de la industria, incluyendo la creación de procedimientos almacenados y triggers que automaticen la lógica del negocio.

Confiamos en que esta actividad permitirá consolidar las habilidades técnicas y desarrollar el criterio profesional necesario para destacar en el campo de la administración de datos.

Objetivos Específicos

- Comprender la instalación y configuración de diferentes servicios y características de Sql Server
- Comprender la instalación de Calidad de Datos con Data Quality Services (DOS)

Orientaciones Específicas

Un administrador de bases de datos no solo ejecuta comandos; piensa en el futuro. Antes de crear una tabla, se pregunta: ¿este tipo de datos es el más eficiente? ¿Esta restricción evitará datos inconsistentes? ¿Necesitaré un índice aquí? ¿Qué pasa si mañana necesito auditar los cambios? Esta evaluación ha sido diseñada para fomentar esa mentalidad crítica y preventiva.

En la sección siguiente se presenta el esquema de trabajo independiente con las fechas establecidas. Se recomienda planificar el tiempo considerando las siguientes fases:

1. Análisis y diseño (comprensión del problema y modelado).
2. Implementación del script (codificación y pruebas iniciales).
3. Desarrollo de procedimientos y triggers (lógica de negocio).
4. Documentación y entrega (informe final).

La actividad deberá entregarse en formato digital (PDF) a través del aula virtual, incluyendo:

- El script SQL completo, profusamente comentado, que refleje el diseño final con todas las mejoras implementadas.
- Un informe narrativo que explique las decisiones de diseño, las mejoras propuestas y la justificación técnica de cada una.
- Evidencias gráficas (capturas de pantalla) de la ejecución exitosa del script y de las pruebas realizadas con procedimientos y triggers.

Se valorará especialmente la originalidad y pertinencia de las mejoras propuestas, así como la claridad de la justificación. No se trata de hacer lo mínimo, sino de demostrar criterio profesional.

Esquema de Trabajo Independiente

Semana	Actividades	Inicio	Finalización (Entrega)
#01	Sumativa N°1: Implementación de base de datos universitaria con procedimientos almacenados y triggers.	07/02/2026	07/03/2026

Actividades de Aprendizaje

Implementación de base de datos universitaria con procedimientos almacenados y triggers.

La Universidad Nacional Héroes de San José de las Mulas está modernizando sus sistemas de información. Usted ha sido contratado como administrador de bases de datos para diseñar e implementar el núcleo del nuevo sistema académico. La universidad le ha proporcionado los requisitos funcionales básicos, pero espera que usted, como profesional, proponga mejoras que optimicen el rendimiento, garanticen la integridad de los datos y automaticen procesos mediante programación transaccional.

Recursos



[Documentación Unidad I: Instalación y configuración de SQL Server](#)



[Vídeo: Cómo instalar SQL Server 2025](#)

[Vídeo: Instalación y configuración de Master Data Service](#)

[Vídeo: Instalación de SQL Server Integration Services](#)

[Vídeo: Creación catálogo de Integration Services](#)

[Vídeo: Instalación y configuración de Reporting services](#)

[Vídeo: Instalación de SQL Server Data Tools](#)

[Vídeo: Instalación de Clusteres](#)

[Vídeo: Agregación de discos a un cluster existente](#)

Requerimientos funcionales base

La base de datos se llamará AcademicaUNHSJM y debe contener cuatro tablas esenciales con la siguiente estructura mínima:

Tabla Carreras

carrera_id (INT, PK, IDENTITY)

nombre_carrera (VARCHAR(100), NOT NULL)

Tabla Alumnos

alumno_id (INT, PK, IDENTITY)

nombre_completo (VARCHAR(100), NOT NULL)

carnet (VARCHAR(15), NOT NULL, UNIQUE)

telefono (VARCHAR(15))

email (VARCHAR(100), NOT NULL)

carrera_id (INT, FK -> Carreras.carrera_id, NOT NULL)

Tabla Cursos

curso_id (INT, PK, IDENTITY)

nombre_curso (VARCHAR(100), NOT NULL)

creditos (INT, NOT NULL)

carrera_id (INT, FK -> Carreras.carrera_id, NOT NULL)

Tabla Inscripciones

inscripcion_id (INT, PK, IDENTITY)

alumno_id (INT, FK -> Alumnos.alumno_id, NOT NULL)

curso_id (INT, FK -> Cursos.curso_id, NOT NULL)

fecha_inscripcion (DATE, NOT NULL)

Desafío profesional: Mejoras obligatorias a implementar

Usted debe implementar la base de datos incorporando al menos TRES de las siguientes mejoras profesionales (puede incluir más, pero debe justificar cada una). Las mejoras deben estar claramente documentadas en el script y en el informe.

CATEGORÍA	MEJORA PROFESIONAL	JUSTIFICACIÓN ESPERADA
DISEÑO DE TABLAS	Agregar columnas de auditoría: <code>fecha_creacion</code> (DATETIME DEFAULT GETDATE()), <code>fecha_modificacion</code> (DATETIME), <code>usuario_creacion</code> (VARCHAR(50)), <code>activo</code> (BIT DEFAULT 1).	Permite trazabilidad y bajas lógicas sin eliminar registros históricos.
RESTRICCIONES AVANZADAS	Implementar restricciones <code>CHECK</code> validadas: email debe contener '@', créditos entre 1 y 5, carnet con formato específico (ej. <code>^[A-Z]{1}[0-9]{5}\$</code>) mediante <code>CHECK</code> con patrón).	Garantiza la calidad de los datos en el origen, evitando basura.
ÍNDICES ESTRATÉGICOS	Crear índices no agrupados en columnas de búsqueda frecuente: <code>Alumnos.carnet</code> , <code>Cursos.carrera_id</code> , <code>Inscripciones.alumno_id</code> .	Mejora el rendimiento de las consultas sin afectar la integridad.
CLAVES ÚNICAS COMPUESTAS	En <code>Inscripciones</code> , agregar una restricción <code>UNIQUE</code> sobre (<code>alumno_id</code> , <code>curso_id</code>) para evitar inscripciones duplicadas.	Refleja una regla de negocio real: un alumno no puede inscribirse dos veces al mismo curso.
VALORES POR DEFECTO INTELIGENTES	En <code>Inscripciones.fecha_inscripcion</code> , usar <code>DEFAULT GETDATE()</code> para que se registre automáticamente la fecha actual.	Reduce errores de ingreso y automatiza procesos.
INTEGRIDAD REFERENCIAL CON ACCIONES	En las claves foráneas, definir <code>ON DELETE CASCADE</code> o <code>ON DELETE SET NULL</code> según corresponda, justificando la decisión.	

Desarrollo de procedimientos almacenados

Debe crear al menos TRES procedimientos almacenados que automaticen operaciones comunes:

1. sp_InsertarAlumno

- Parámetros: todos los campos de Alumnos (excepto alumno_id que es IDENTITY).
- Debe validar que el carrera_id exista antes de insertar.
- Debe manejar errores con TRY...CATCH y devolver mensajes claros.
- Opcional: Generar el carnet automáticamente basado en la carrera y un correlativo.

2. sp_InscribirAlumno

- Parámetros: alumno_id, curso_id.
- Debe verificar que el alumno y el curso existan.
- Debe verificar que el curso pertenezca a la carrera del alumno (un alumno no puede inscribirse en cursos de otra carrera).
- Debe evitar inscripciones duplicadas (usando la restricción o validación manual).
- Insertar en Inscripciones con la fecha actual.

3. sp_ListarInscripcionesPorAlumno

- Parámetro: alumno_id (o carnet).
- Debe devolver un conjunto de resultados con: nombre del alumno, nombre del curso, créditos, fecha de inscripción.
- Si el alumno no existe, debe devolver un mensaje de error o un conjunto vacío.

Desarrollo de triggers

Debe crear al menos DOS triggers que automaticen reglas de negocio:

1. trg_ActualizarFechaModificacion (sobre tabla Alumnos o Cursos)
 - Trigger AFTER UPDATE que actualice automáticamente la columna fecha_modificacion con GETDATE() cuando se modifique un registro.
 - Requiere haber implementado la columna de auditoría.
2. trg_ValidarCreditosCurso (sobre tabla Cursos)
 - Trigger INSTEAD OF INSERT o AFTER INSERT que valide que los créditos del curso sean positivos y no excedan un máximo (ej. 5).
 - Si no cumple, debe rechazar la inserción con un mensaje de error claro.

Alternativa: Un trigger que impida eliminar cursos que tengan inscripciones asociadas.

Entregables concretos

1. Script maestro (UniversidadUNH_Script.sql). Debe ser re-ejecutable (usar IF EXISTS para eliminar objetos si existen). Debe incluir:
 - Creación de la base de datos (con opciones personalizadas de archivos).
 - Creación de tablas con todas las mejoras implementadas.
 - Creación de índices y restricciones adicionales.
 - Creación de procedimientos almacenados.
 - Creación de triggers.
 - Inserción de datos de prueba (mínimo 3 carreras, 5 alumnos por carrera, 4 cursos por carrera, inscripciones de ejemplo).
 - El script debe estar profusamente comentado, explicando cada mejora y decisión técnica.
2. Informe técnico (Informe_Sumativa1.pdf)

- Portada, introducción y objetivos.
- Explicación detallada de las mejoras implementadas (por qué se eligieron, qué problema solucionan).
- Justificación de la lógica implementada en procedimientos y triggers.
- Capturas de pantalla que evidencien:
 - La ejecución exitosa del script completo.
 - Pruebas de los procedimientos almacenados (ejecutando `sp_InsertarAlumno`, `sp_InscribirAlumno`, etc. con casos válidos e inválidos).
 - Pruebas de los triggers (mostrando cómo actúan ante inserciones/actualizaciones).
- Conclusiones y recomendaciones para una futura expansión del sistema.

Criterios de Evaluación

La actividad sumativa se evaluará sobre 100 puntos, distribuidos según la siguiente tabla:

CRITERIO	DESCRIPCIÓN DEL DESEMPEÑO ESPERADO	PORCENTAJE
DISEÑO Y ESTRUCTURA DE LA BASE DE DATOS (25%)	Tablas correctamente definidas con tipos de datos apropiados. Claves primarias y foráneas bien establecidas. Implementación de al menos tres mejoras profesionales justificadas. Uso adecuado de restricciones (UNIQUE , CHECK , DEFAULT , NOT NULL).	25%
CALIDAD DEL SCRIPT T-SQL (20%)	Script completo, re-ejecutable, con comentarios claros que explican las decisiones técnicas. Organización lógica (primero tablas, luego datos, luego objetos). Buenas prácticas de formato y legibilidad.	20%
PROCEDIMIENTOS ALMACENADOS (20%)	Creación de al menos tres procedimientos funcionales. Manejo de parámetros,	20%

	validaciones, control de errores con TRY...CATCH . Código eficiente y bien comentado. Pruebas exitosas documentadas.	
TRIGGERS (15%)	Creación de al menos dos triggers que resuelvan problemas reales (auditoría, validaciones, reglas de negocio). Correcta implementación y justificación de su uso. Evidencia de funcionamiento.	15%
DOCUMENTACIÓN Y JUSTIFICACIÓN DE MEJORAS (20%)	Informe estructurado y profesional. Explicación clara de las mejoras implementadas y por qué son valiosas. Capturas de pantalla relevantes y bien presentadas. Capacidad crítica y de comunicación.	20%